

Session & Token Hijacking

What session hijacking is

Attacker obtains a victim's *identifier* (session cookie, JWT, bearer token) and impersonates them – often without knowing the password.

Attack vectors

Vector	How
Cookie theft	XSS → <code>document.cookie</code> → send high-value token off-site
Session fixation	Attacker <i>sets</i> the session ID; victim authenticates the attacker's session; attacker then uses it
CSRF-adjacent	Victim, while logged in, submits a crafted request carrying their token
Replay / theft via MITM	Steal the token in transit (if TLS/HTTP-flow mishandled)
JWT weaknesses	alg:none, alg confusion (RS256→HS256), weak secret, missing expiry
Token side-channels	Token in URL (leaks to Referer/logs), in localStorage where XSS reads it

JWT attack micro-quick-reference

```
# try alg:none
{"alg":"none","typ":"JWT"} . <payload> . "" of the jwt.io form

# algorithm confusion RS256 -> HS256 using the PUBLIC key as the HMAC secret
# weak secret crack: hashcat -m 16500 jwt.txt rockyou.txt

# missing 'exp' -> never expires; try future times
```

Countermeasures (beyond 'how' the attack works)

`HttpOnly` cookies for the session token (XSS can't read them)

`Secure + SameSite` (Lax/Strict) attributes guard CSRF + transit

`Short-lived + refresh rotation`: token leaks die fast

`Bind token to risk`: re-auth (MFA) on privilege change; new session ID on login (fixation)

`Never in URL`; keep JWT secrets out of source

`WAF/filter` on auth headers; monitor abnormal token usage

Detection mindset

Sudden geography/user-agent change + a valid token = replay

Same token from two IPs at once

Token without logout/invalidate pattern – session management logs triage this

Token craft + session attributes + rotation is a triangle; your countermeasures must cover all three edges.

[← Layer-2 & Switch Attacks](#)

[DHCP Abuses & DNS Tunneling →](#)